

COMP202 Week 9 - solutions

Question 1 (Part a of CLRS, Exercise 34-1). *An independent set of a graph $G = (V, E)$ is a subset $V' \subseteq V$ of vertices such that each edge in E is incident on at most one vertex in V' . The independent-set problem is to find a maximum-size independent set in G .*

Formulate a related decision problem for the independent-set problem, and prove that it is NP-complete. You may assume NP-completeness of the Clique problem (see the class materials).

Solution: Let us define INDSET to be the decision version of the independent set problem. Just as the decision version of the Clique problem, INDSET takes as input a graph $G = (V, E)$ and a parameter k . The question to be solved is: “Does G have an independent set of size at least k ?” Recall that we need to show things to show that a problem is NP-complete: we need to show that it is in NP, and also that a known NP-complete problem reduces to it (i.e., it is NP-hard).

First let us observe that $\text{INDSET} \in \text{NP}$. For a YES-instance G (i.e., G has an independent set of size at least k), an efficient certificate is a subset of vertices $V' \subseteq V$ such that $|V'| \geq k$ and V' is an independent set. An efficient certifier simply checks that V' is actually an independent set. Hence, $\text{INDSET} \in \text{NP}$.

We now show that the decision version of the Clique problem reduces to INDSET. Consider an input G, k to the Clique problem (thus, the task is to determine if there exists a clique in G with at least k vertices). In polynomial time, we can construct the graph $\overline{G}$ which is defined by “flipping” the status of all edges/non-edges in G . That is, if (u, v) is an edge in G then it is not an edge in $\overline{G}$. On the other hand, if (u, v) is not an edge in G , then it is an edge in $\overline{G}$. Note that a subset of vertices V' is a clique in G if and only if it is an independent set in $\overline{G}$. Thus, the answer to the original instance G, k to the Clique problem is YES if and only if the answer to INDSET on input $\overline{G}, k$ is YES. This completes the reduction, and hence INDSET is NP-hard.

Question 2 (Parts a and c of CLRS, Exercise 34-2). *Bonnie and Clyde have just robbed a bank. They have a bag of money and want to divide it up. For the following scenarios, either give a polynomial-time algorithm, or prove that the problem is NP-complete. The input in each case is a list of the n items in the bag, along with the value of each. Assume that integers can be added and multiplied in constant time.*

- The bag contains n coins, but only 2 different denominations: some coins are worth x pounds and some are worth y pounds. Bonnie and Clyde wish to divide the money exactly.
- The bag contains n checks, which are, in an amazing coincidence, made out to “Bonnie or Clyde” (i.e., either one of them can cash it). They wish to divide the checks so that they each get the exact same amount of money.

Solution:

- There is a polynomial-time algorithm for this setting, described below. Suppose there are n_1 coins worth x pounds and n_2 coins worth y pounds. Suppose there exists an equal division in which Bonnie gets b_x many x -coins and b_y many y -coins, and Clyde gets c_x many x -coins and c_y many y -coins. Here, $b_x + c_x = n_1$ and $b_y + c_y = n_2$. Because the money is equally divided, this means

$$b_x \cdot x + b_y \cdot y = c_x \cdot x + c_y \cdot y.$$

There are only $n_1 + 1 \leq n + 1$ choices for b_x (integers between 0 and n_1), and $n_2 + 1 \leq n + 1$ choices for b_y . Thus, there are at most $(n + 1)^2 = O(n^2)$ choices for the pair (b_x, b_y) . Each choice of b_x, b_y fixes $c_x = n_1 - b_x$ and $c_y = n_2 - b_y$. For each of the $O(n^2)$ choices of pairs (b_x, b_y) , we can check if

$$b_x \cdot x + b_y \cdot y = (n_1 - b_x) \cdot x + (n_2 - b_y) \cdot y.$$

Since each check takes constant time, we can find an equal division (or output that no such equal division exists) by going over all $O(n^2)$ choices in $O(n^2)$ time.

- We show that the problem is NP-complete in this setting. Clearly it is in NP: an efficient certificate is the information about which check goes to who. An efficient certifier simply needs to verify that this split of checks splits the money evenly between Bonnie and Clyde.

It remains to show that the problem is NP-hard. This is exactly the Subset-Sum problem (where the target is not 0, but half of the total sum).

Question 3. Consider the problem of graph 2-colouring, defined as follows: The input is a graph $G = (V, E)$. The goal is to assign to every vertex in V , one of two colours (say red and blue), such that no two vertices sharing an edge has the same colour. Output “Yes” if such a colouring exists, and “No” if it does not exist.

Give a polynomial time algorithm for this problem.

Solution: Suppose we are trying to colour an input graph G using the 2 colours R (red) and B (blue). Let us assume that the graph is connected to begin with (which can be done using DFS/BFS). If it is not connected, then we just need to solve the 2-colouring problem on each of the connected components.

Start with an arbitrary vertex v and colour it R. Now we repeat the following loop until all vertices are coloured: While there is an uncoloured vertex w adjacent to a coloured one x , colour w with the opposite colour as that of x .

Once all vertices are coloured, check all edges one at a time to see if they are coloured appropriately (i.e., they don't have the same colour). If the colouring is legal, then this gives a 2-colouring. The running time of this algorithm is essentially the running time of BFS/DFS, along with the final check that takes $O(|E|)$ time, giving a total running time of $O(|V| + |E|)$.

Question 4. *Prove that if $\text{co-NP} \subsetneq \text{NP}$, then $\text{P} \neq \text{NP}$.*

Solution: Let L be a problem that is in NP but not in co-NP. In other words,

$$L \notin \text{NP} \cap \text{co-NP}.$$

We saw in class that $\text{P} \subseteq \text{NP} \cap \text{co-NP}$. This means $L \notin \text{P}$. But we started with the assumption that $L \in \text{NP}$. Thus, $\text{P} \neq \text{NP}$. **Remark:** A similar argument can be used to show that if $\text{co-NP} \neq \text{NP}$, then $\text{P} \neq \text{NP}$.

Question 5 (Decision vs. Search for CNF-SAT). *Suppose there is a polynomial-time algorithm for the decision problem CNF-SAT as discussed in class (this is just 3-SAT but the clauses could have any number of literals, not necessarily 3). Show that there is also a polynomial-time algorithm to find a satisfying assignment of a given (satisfiable) CNF formula ϕ .*

Solution: Let $x_1, x_2, \dots, x_n$ be the variables in a given CNF-SAT instance ϕ . Let $\mathcal{A}$ be a polynomial-time algorithm for solving CNF-SAT. First check, using $\mathcal{A}$ if ϕ is satisfiable. If it isn't, output that the formula is unsatisfiable. If it is, proceed to the next step.

Let $\phi|_{x_1=0}$ be the formula obtained by setting all occurrences of x_1 in ϕ to 0. Similarly, define $\phi|_{x_1=1}$. Constructing these formulas can be done in polynomial time since it only involves fixing values of variables. Since ϕ was satisfiable, at least one of $\phi|_{x_1=0}$ or $\phi|_{x_1=1}$ is satisfiable. We can use $\mathcal{A}$, once for each of these formulas, to check which one of them is satisfiable. Suppose $\phi|_{x_1=0}$ was satisfiable (the argument if $\phi|_{x_1=1}$ is similar). Let $\phi' = \phi|_{x_1=0}$. Observe that we have applied $\mathcal{A}$ twice and obtained a new formula on one less variable (since we've fixed the value of x_1) that is satisfiable. Continuing this process and fixing each variable, we get an algorithm that fixes one variable in each step using two invocations of $\mathcal{A}$. This process ends in n steps, at the end of which we have an assignment to all n variables. The invariant

maintained at each step is that we are always dealing with a satisfiable formula. Since each step only requires a polynomial overhead (for fixing the variables to obtain a new formula) and two invocations of $\mathcal{A}$ (which is a polynomial-time algorithm by assumption) per step, this whole process is a polynomial-time algorithm, and returns a satisfying assignment.